COMP2021 Object-Oriented Programming

CLEVIS

Command Line Vector Graphics Software

Group 18

Meng Zihan **Huang Junhao** **Luo Yeyushan** **He Liuying**

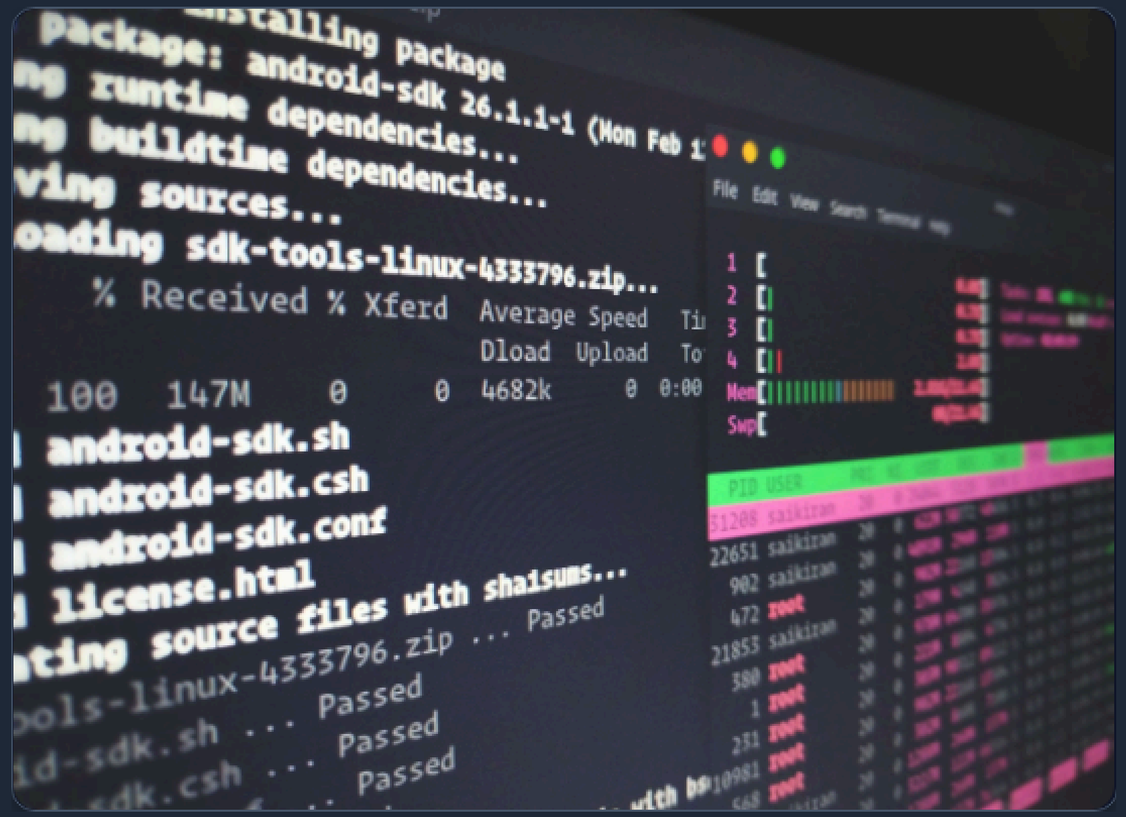
Introduction

Project Overview

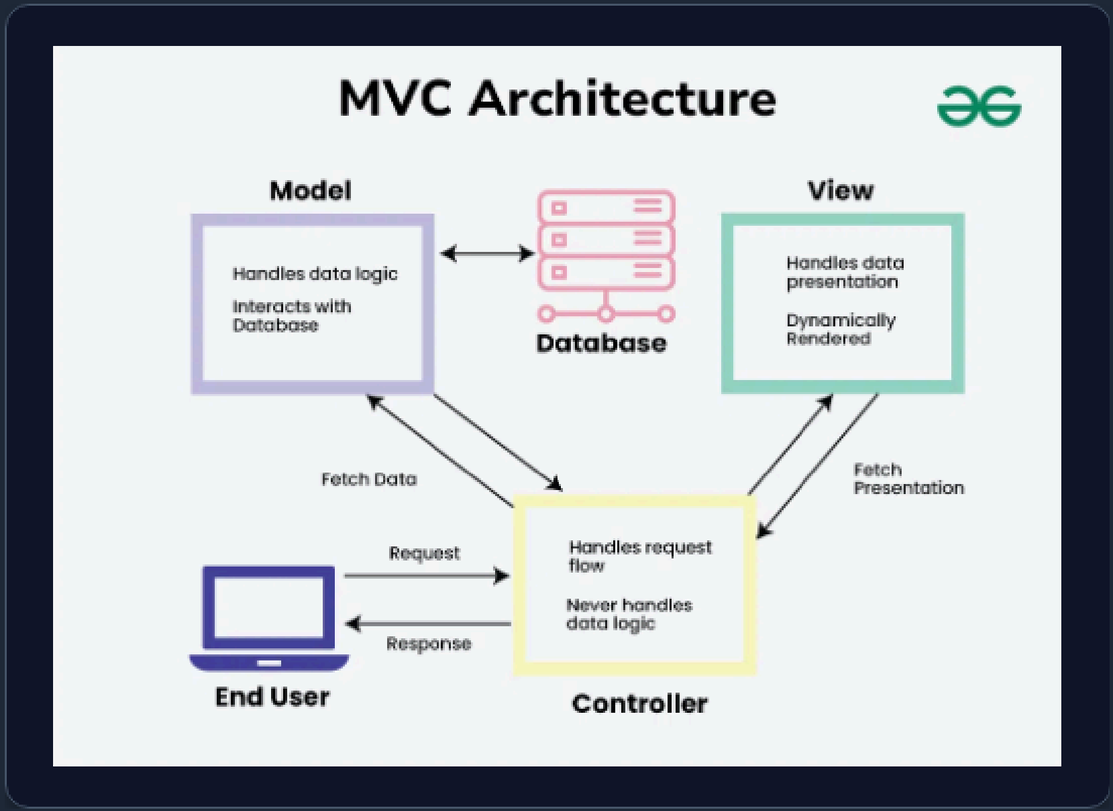
Clevis is a command-line vector graphics software that enables users to create, manipulate, and manage geometric shapes through intuitive text commands.

We have implemented core features including shape creation, grouping (Composite Pattern), Undo/Redo (Memento Pattern), and comprehensive error logging.

```
clevis> line l1 15 30 23 12
clevis> Line 'l1' created successfully.
```



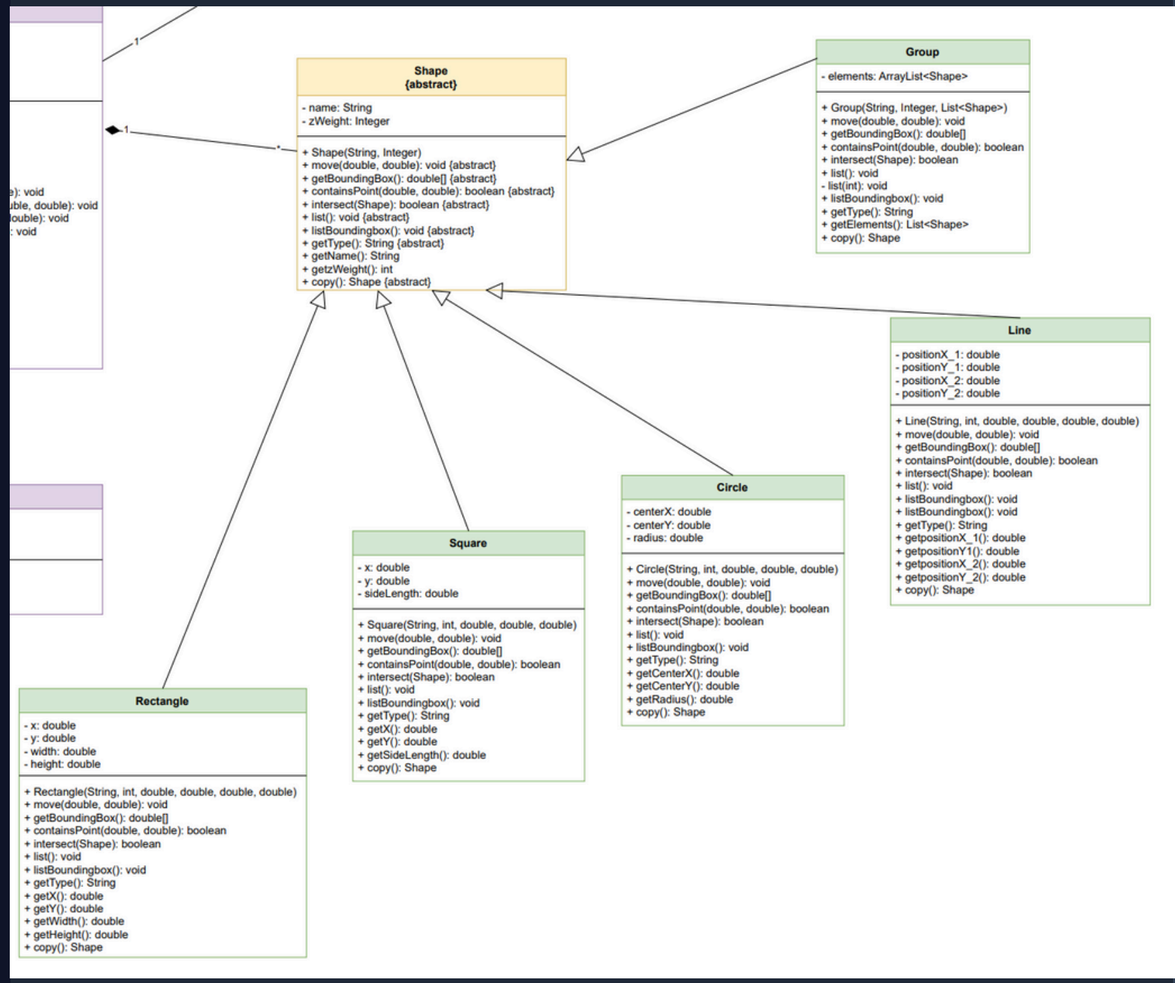
System Architecture (MVC)



- **Model (Logic):** ShapeManager and all shape classes, handling core logic and data management.
- **View (Output):** CommandLogger, which records all operations into HTML and text files.
- **Controller (Input):** Commander for command processing and Clevis for workflow coordination.
- **Separation of Concerns:** This ensures that input parsing is decoupled from business logic.

Inheritance & Polymorphism

- All concrete components (Line, Rectangle, Square, Circle, Group) inherit directly from the abstract Shape class.
- System interacts only with the generic Shape abstraction.
- Benefit: Decouples command logic (e.g., move, list) from specific shape types.
- Execution: The actual object instance ensures the correct behavior is executed automatically.



The Square vs. Rectangle

- Math vs. Code: Mathematically, a square is a rectangle; programmatically, this creates a data representation conflict.
- Inheritance would require awkward constructors with redundant arguments.
- Risk of exposing methods allowing width and height to vary independently.

```
public Square(String name, int zWeight, double x, double y, double sideLength)
    super(name, zWeight);
    this.x = x;
    this.y = y;
    this.sideLength = sideLength;
}
```

```
public Rectangle(String name, int zWeight, double x, double y, double width, double height) {
    super(name, zWeight);
    this.x = x;
    this.y = y;
    this.width = width;
    this.height = height;
}
```

What OOP Principles Did We Use?

except Inheritance/Hierarchy, Modularity/Polymorphism mentioned before...

Abstraction

- **Users don't need to know the complex math behind these operations**

- **Real implementation complexity is hidden:**

```java
//REQ10
@Override  2个用法
public void move(double dx,double dy){
    this.positionX_1+=dx;
    this.positionY_1+=dy;
    this.positionX_2+=dx;
    this.positionY_2+=dy;
}
```

```java
public abstract class Shape {
public abstract void move(double dx, double dy);
public abstract double[] getBoundingBox();
public abstract boolean containsPoint(double px, double py);
public abstract boolean intersect(Shape other);
public abstract void list();
public abstract void listBoundingbox();
public abstract String getType();
```

```java
//getters
public String getName() {
    return name;
}

public int getzWeight() {
    return zWeight;
}
```

What OOP Principles Did We Use?

except Inheritance/Hierarchy, Modularity/Polymorphism mentioned before...

Encapsulation

- **Critical data structures are private**
  - **Private: internal identity**
  - **Private: rendering order**
  - **Public getters for controlled access**

No Encapsulation:

- **Defining clear interaction points**
  - **Public contract - what shapes "can do"**

```
public abstract class Shape {  41 个用法  5 个继承者

    private String name;  2 个用法
    private Integer zWeight;  2 个用法

    public Shape(String name,Integer zWeight){  5 个用法
        this.name=name;
        this.zWeight=zWeight;
    }
```

```
public abstract void move(double dx, double dy);
public abstract double[] getBoundingBox();
public abstract boolean containsPoint(double px, double py);
public abstract boolean intersect(Shape other);
public abstract void list();
public abstract void listBoundingbox();
public abstract String getType();
```

OOP Benefit: Reusability

- We decided to implement Shape as an abstract class rather than an interface. This was a deliberate choice to improve code **reusability**.
- Since every single shape in our system—whether it's a simple line or a complex group—requires a unique name and a z-weight for layering, using an abstract class allowed us to define these common attributes once in the parent class.
- If we had used an interface, we would have been forced to duplicate this code in every single subclass, which violates the DRY (Don't Repeat Yourself) principle.

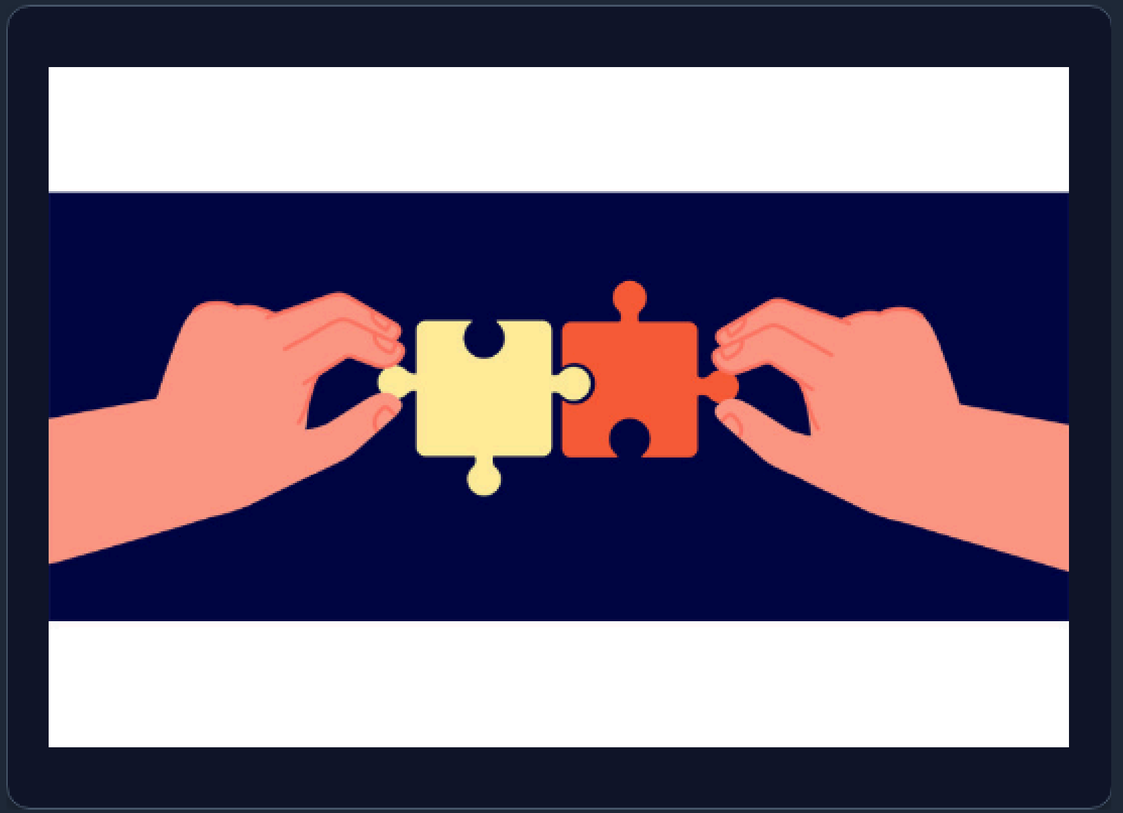
```
public abstract class Shape {  41 个用法  5 个继承者

    private String name;  2 个用法
    private Integer zWeight;  2 个用法

    public Shape(String name,Integer zWeight){  5 个用法
        this.name=name;
        this.zWeight=zWeight;
    }
```

OOP Benefit: Scalability

Open/Closed Principle

The system is open for extension but closed for modification.

- **New Shapes: If we need to add a "Triangle", we simply create “Triangle extends Shape” and implement the abstract methods. We do *not* need to change the Group logic or the move command logic.**
- **New Commands: The Commander structure allows us to easily plug in new command parsers *without* disrupting the Model layer.**



Bonus: Undo & Redo

Initial version
-Run the “opposite command”

For example:

move a 99 99 -> move a -99 -99

delete b -> create b

group g1 c1 c2 -> ungroup g1

....

However,
This will cause many problems!

Although we can reuse our code easily, for some command, it is a not wise choice...

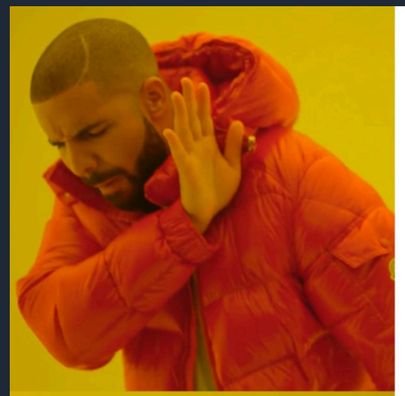
eg: Group may contain Group!

It is a hard work to create all the deleted shapes and build their original group correctly if we want to redo a “delete group”

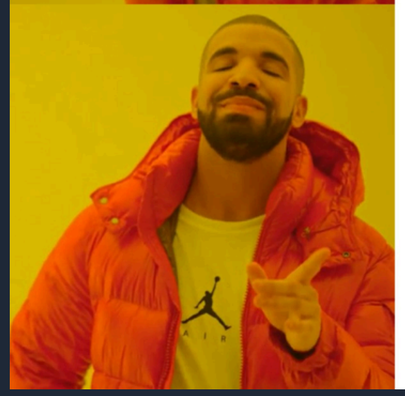
*Nested Groups be like:

Bonus: Undo & Redo

Let’s jump out of this logic trap!
-Does how we made it matters?



Process



Output

*Output-Oriented Programming

We noticed that all the shape informations are stored in few data structures

```
private final Map<String,Shape> ShapesMap;//name-Shapes
private final List<String> WeightList;//z-Order (name)
private final Set<String> Locker;
```

- ShapesMap: store all the shapes and their name
- WeightLists: store the order of shapes
- Locker: the shapes that locked in group and can not interact with

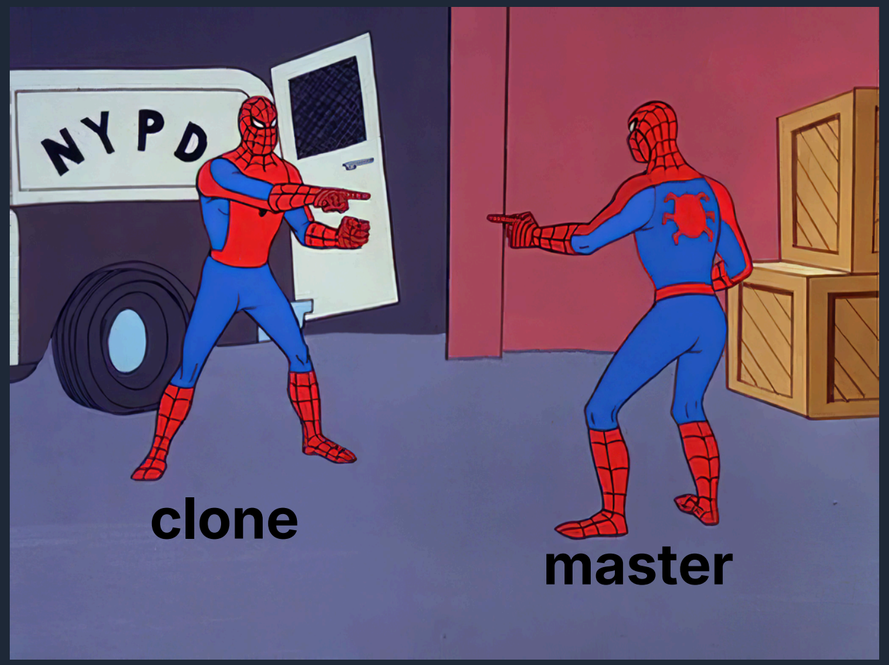
And all the commands need to base on the information that these data structures store

In this case, we only need to backup these data structures before user enter the commands that can be undo. (Redo is rather similar) so we create Cache to backup.

```
private class Cache{
    private final Map<String, Shape> shapesBackup;
    private final List<String> weightListBackup;
    private final Set<String> lockerBackup;

    public Cache(){
        this.shapesBackup=new HashMap<>();
        this.weightListBackup=new ArrayList<>();
        this.lockerBackup=new HashSet<>();
```

Bonus: Undo & Redo

After deal with the common datas, we found that private data of shapes will change in new command (positions for example), so the shapes in backups can not be the original one.
Instead, we clone them.




```java
private Shape cloneShape(Shape master) {
    //创建母版的克隆图形
    if (master instanceof Rectangle){
        Rectangle clone = (Rectangle) master;
        return new Rectangle(clone.getName(),clone.getzWeight(),clone.getX(),clone.getY(),clone.getWidth(),clone.getHeight());
    }
    else if (master instanceof Square){
        Square clone = (Square) master;
        return new Square(clone.getName(),clone.getzWeight(),clone.getX(),clone.getY(),clone.getSideLength());
    }
    else if (master instanceof Circle){
        Circle clone = (Circle) master;
        return new Circle(clone.getName(),clone.getzWeight(),clone.getCenterX(),clone.getCenterY(),clone.getRadius());
    }
    else if (master instanceof Line){
        Line clone = (Line) master;
        return new Line(clone.getName(),clone.getzWeight(),clone.getPositionX_1(),clone.getPositionY_1(),clone.getPositionX_2(),clone.getPositionY_2()
    }
    else if (master instanceof Group){
        Group group = (Group) master;
        List<Shape> elements = new ArrayList<>();
        for(Shape element:group.getElements()) elements.add(cloneShape(element));
        return new Group(group.getName(),group.getzWeight(),elements);
    }
    return null;
}
```

That is how we store the private datas

Bonus: Undo & Redo

Consider Operation method of redo/undo, we found that the command obey FILO law, which is also the feature of Stack.

```
private final Stack<Cache> undoStack = new Stack<>();
private final Stack<Cache> redoStack = new Stack<>();
```

Stack Operations

Through these simple operations, we can easily solve the problem of redo/undo

```
public void restore(){
    ShapesMap.clear();
    WeightList.clear();
    Locker.clear();

    ShapesMap.putAll(shapesBackup);
    WeightList.addAll(weightListBackup);
    Locker.addAll(lockerBackup);
}
```

*rewrite

```
public void undo(){
    if(undoStack.isEmpty()){
        System.out.println(x: "clevis> Nothing to undo.");
        return;
    }

    // 保存当前状态到redo栈
    redoStack.push(new Cache());
    Cache cache=undoStack.pop();
    cache.restore();
    System.out.println(x: "clevis> Undo successful.");
}
```

*undo

```
public void redo(){
    if(redoStack.isEmpty()){
        System.out.println(x: "clevis> Nothing to redo.");
        return;
    }

    // 保存当前状态到undo栈
    undoStack.push(new Cache());
    Cache cache = redoStack.pop();
    cache.restore();
    System.out.println(x: "clevis> Redo successful.");
}
```

*redo

Thank You!

Group 18